SKYCAST

Plateforme de gestion météorologique

Une architecture distribuée basée sur les microservices.



MEMBRES DU GROUPE



Maryem Zayani



Maissa Messoussi



Aziz Tombari



Alaa Ben Moussa



Mariem Hamdi

SOMMAIRE

01 Idée générale problématique

02 Principales fonctionnalités

03 Architecture globale proposée

04 Microservices et bases de données

05 Technologies choisies

06 Équipe et répartition des tâches

07 Organisation du dépôt Git

08 Planning prévisionnel et les principaux risques

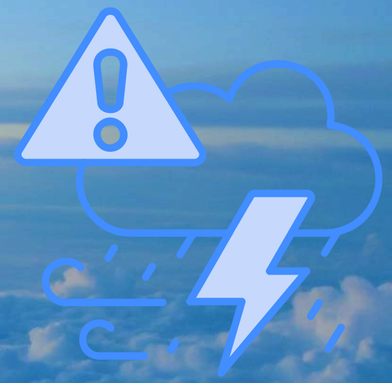
09 MICROSERVICES : CONCEPTION & INTERACTIONS

10 CONCLUSION

Idée générale et problématique

Contexte

Des stations équipées de capteurs collectent températures, humidité, vent et pression. L’application centralise ces données par ville : historiques, statistiques, prévisions et alertes, pour visiteurs, utilisateurs inscrits et administrateurs.





Problématique

Comment exposer des données météo fiables et à jour, tout en prévenant rapidement les abonnés en cas de phénomène dangereux, avec une architecture évolutive et faiblement couplée ?

Principales fonctionnalités

Visiteur

- Consulter les villes — Ville
- Voir les prévisions sur 7 jours — Prévision
- Voir la dernière mesure météo — Relevé
- Consulter les alertes actives — Alerte

Utilisateur

- Gérer son profil — Auth
- S'abonner aux alertes — Alerte
- Voir ses notifications — Alerte
- Consulter les stations — Station
- Voir les historiques et statistiques — Relevé
- Comparer les prévisions et les mesures — Prévision

Administrateur

- Gérer les villes — Ville
- Gérer les stations et capteurs — Station
- Gérer les mesures — Relevé
- Gérer les prévisions — Prévision
- Gérer les alertes et leurs règles — Alerte
- Gérer les comptes — Auth

Acteurs et cas d’utilisation

Visiteur · non authentifié
Consulte les informations publiques : villes, prévisions sur 7 jours, dernière mesure, alertes actives.

Utilisateur (USER) · authentifié
Hérite des droits du visiteur : profil, abonnements, notifications, historique, statistiques, comparaison prévision / mesure.

Administrateur (ADMIN) · authentifié
Gère villes, stations, capteurs, mesures, prévisions, règles d’alerte, alertes et comptes.

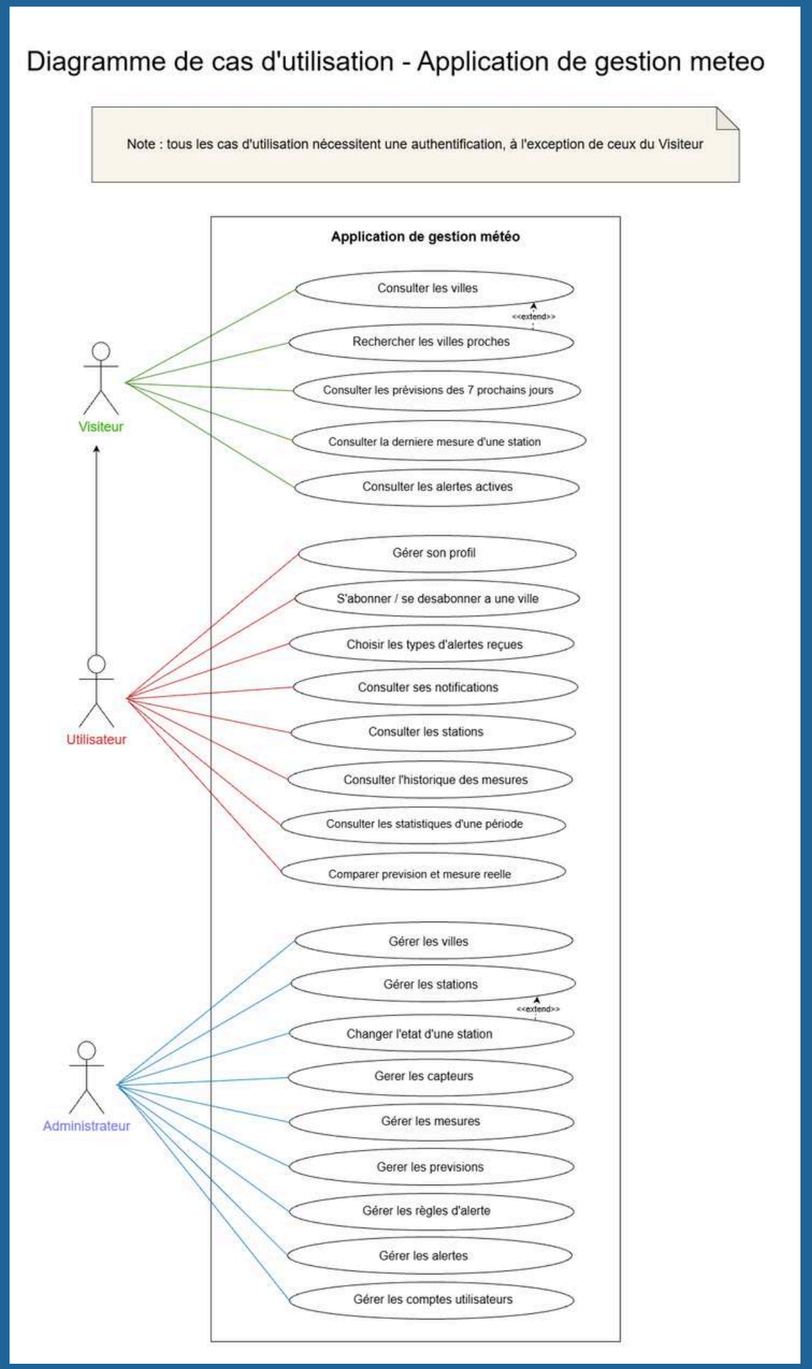
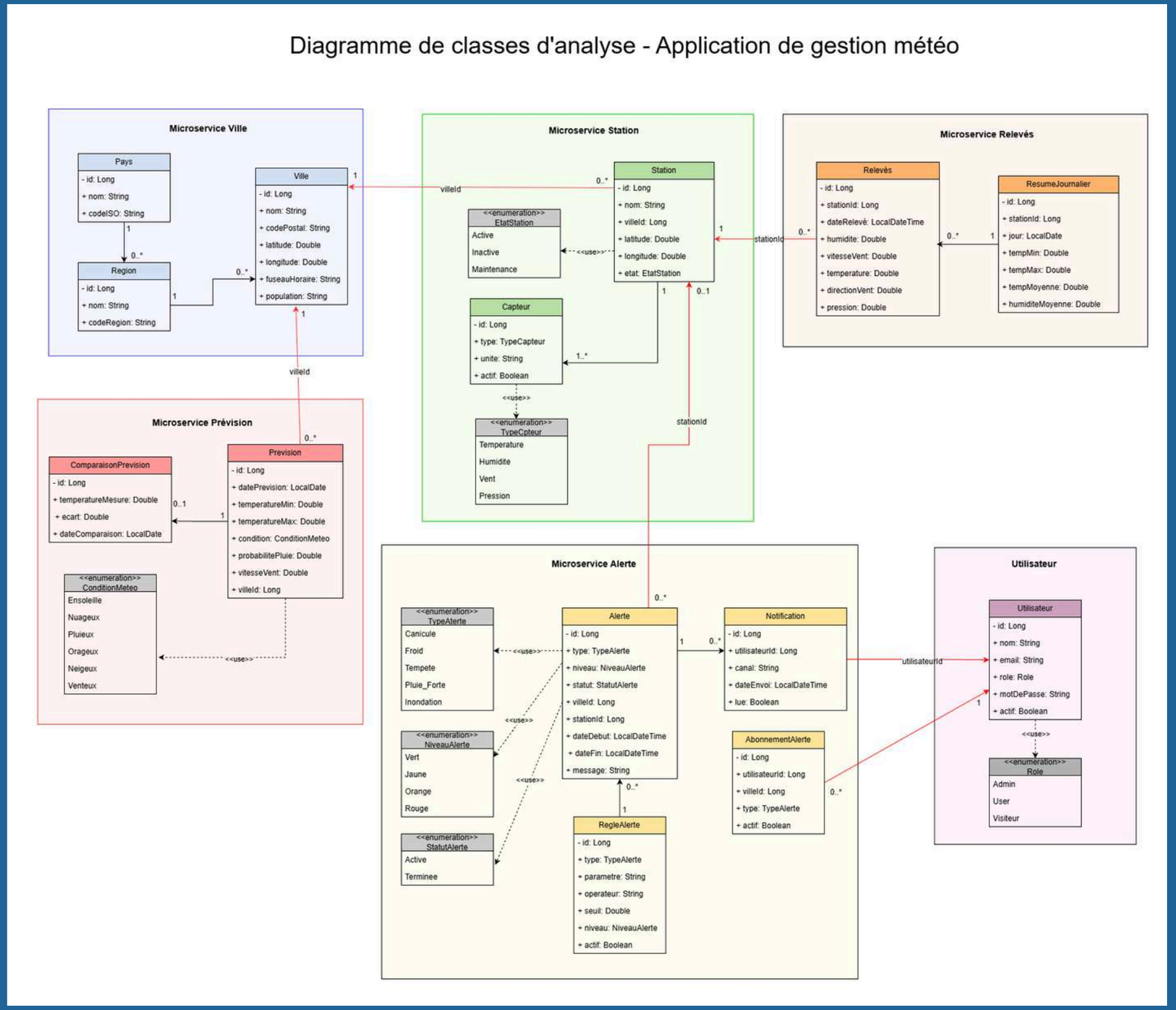
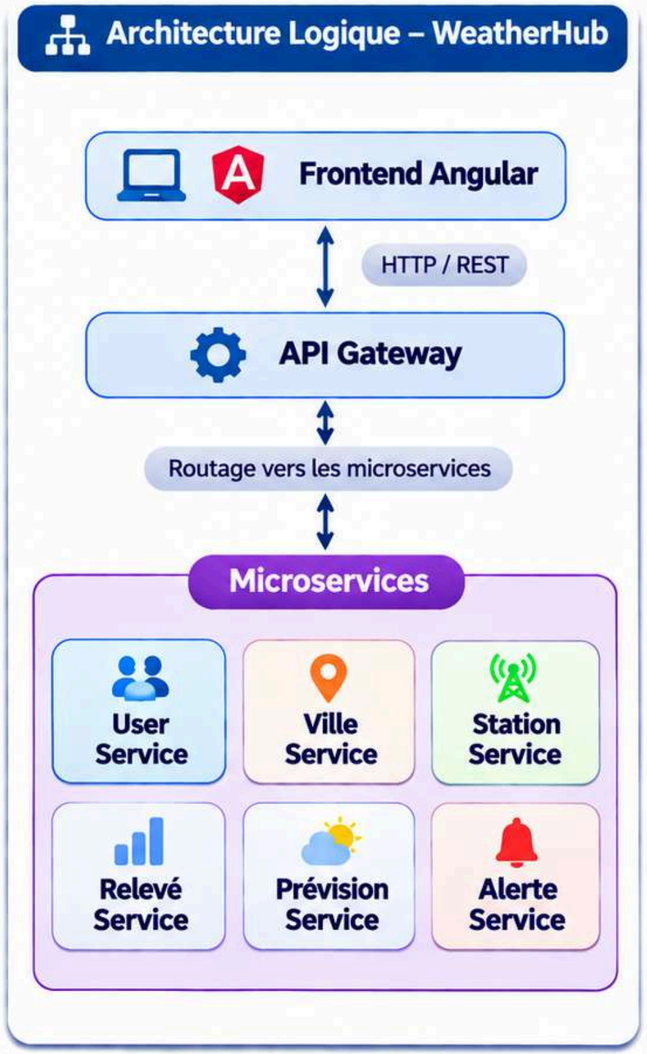
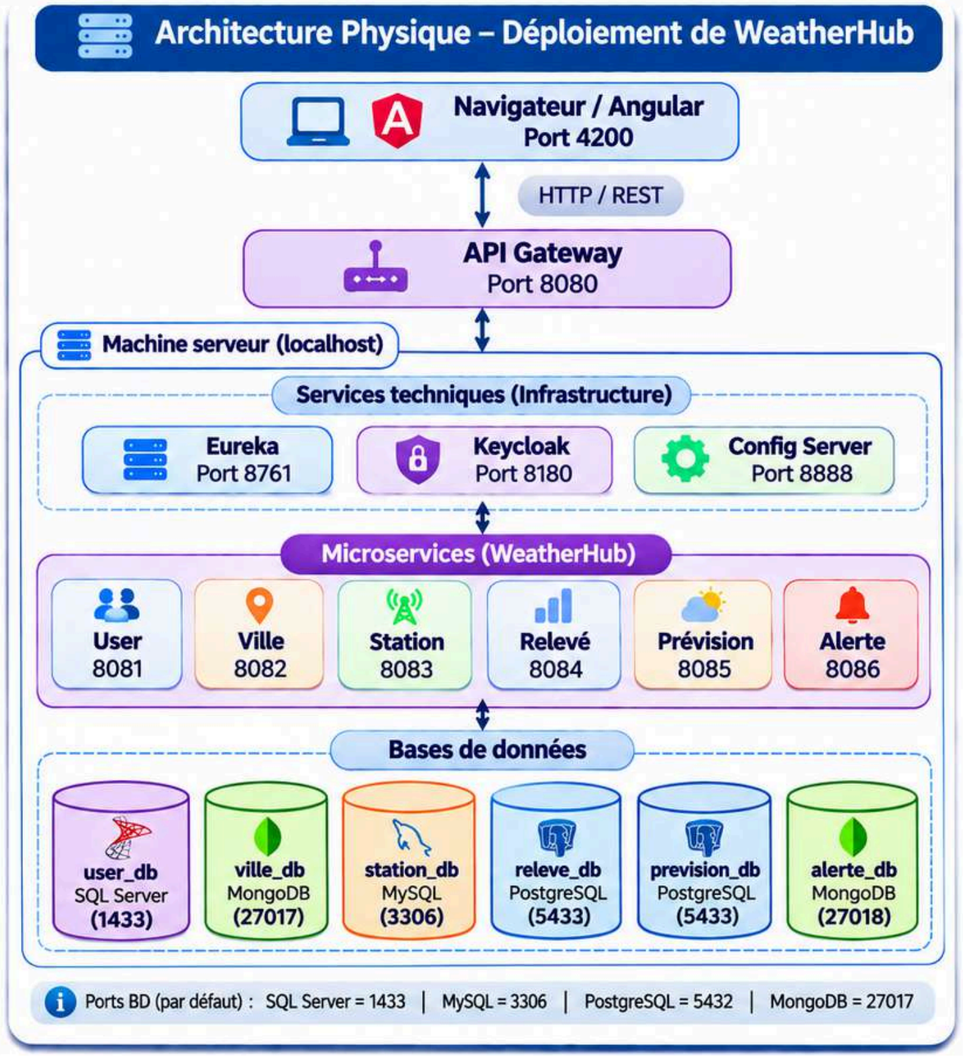


Diagramme de classes d'analyse



Architecture globale





API Gateway · 8080
point d'entrée unique : routage, JWT, rôles, CORS

Eureka · 8761
enregistrement et découverte des services

Keycloak · 8180
gestion des identités et des accès

Config Server · 8888
configuration centralisée

Service commun — Pourquoi Django ?

AUTHENTIFICATION PARTAGÉE / SKYCAST

Django (Python) • Service d’authentification commun à la plateforme

PORT 8081

Sécurité intégrée

Authentification, permissions et protection des données.

Développement rapide

Fonctionnalités prêtes à l’emploi et moins de code.

API faciles à exposer

Intégration possible avec Angular et les services Spring Boot.

Écosystème Python

Framework mature, documentation riche et maintenance facilitée.

Choix retenu : centraliser la gestion des utilisateurs et des accès.

Microservices et bases de données

Ville	**Pays, régions, villes, villes proches** données hiérarchiques, index géospatiaux	**MongoDB · ville_db**	Alaa · 8082
Station	**Stations, capteurs, état** modèle relationnel simple 1-N	**MySQL · station_db**	Mariem · 8083
Relevé	**Mesures, résumés, historique, statistiques** gros volume, agrégations SQL	**PostgreSQL · releve_db**	Aziz · 8084
Prévision	**Prévisions 7 jours, comparaison** données structurées, calculs d'écart	**PostgreSQL · prevision_db**	Maryem · 8085
Alerte	**Règles, alertes, notifications, abonnements** schéma flexible	**MongoDB · alerte_db**	Maissa · 8086
Commun (Auth)	**Utilisateurs, rôles, JWT** ACID, intégrité, sécurité	**SQL Server · user_db**	Équipe · 8081

Technologies choisies

BACKEND

Java · Spring Boot · Spring Data JPA

MICROSERVICES

Spring Cloud Gateway · Eureka · Config Server · OpenFeign

SÉCURITÉ

Keycloak · Spring Security · JWT

BASES DE DONNÉES

SQL Server · MySQL · PostgreSQL · MongoDB

FRONTEND

Angular

DOCUMENTATION D'API

Swagger / OpenAPI

CONTENEURISATION

Docker · Docker Compose

VERSIONNEMENT

Git · GitHub

GESTION DE PROJET

GitHub

TESTS

Postman

Équipe et répartition des tâches

MZ

Maryem Zayani
Prévision Service
port 8085

MM

Maissa Messoussi
Alerte Service
port 8086

AT

Aziz Tombari
Relevé Service
port 8084

AM

Alaa Ben Moussa
Ville Service
port 8082

MH

Mariem Hamdi
Station Service
port 8083

Service commun (Auth) · 8081
développé par toute l'équipe

Parties transverses
Gateway, Eureka, Config Server, Frontend et documentation : réparties équitablement

Microservice Ville



RÔLE ET BASE DE DONNÉES DU MICRO-SERVICE

Le service Ville centralise et gère les données géographiques nécessaires au fonctionnement de l'application.

- **Gestion des pays**
- **Gestion des régions**
- **Gestion des villes**
- **Recherche et consultation des données géographiques**
- **Mise à disposition des informations géographiques aux autres microservices**

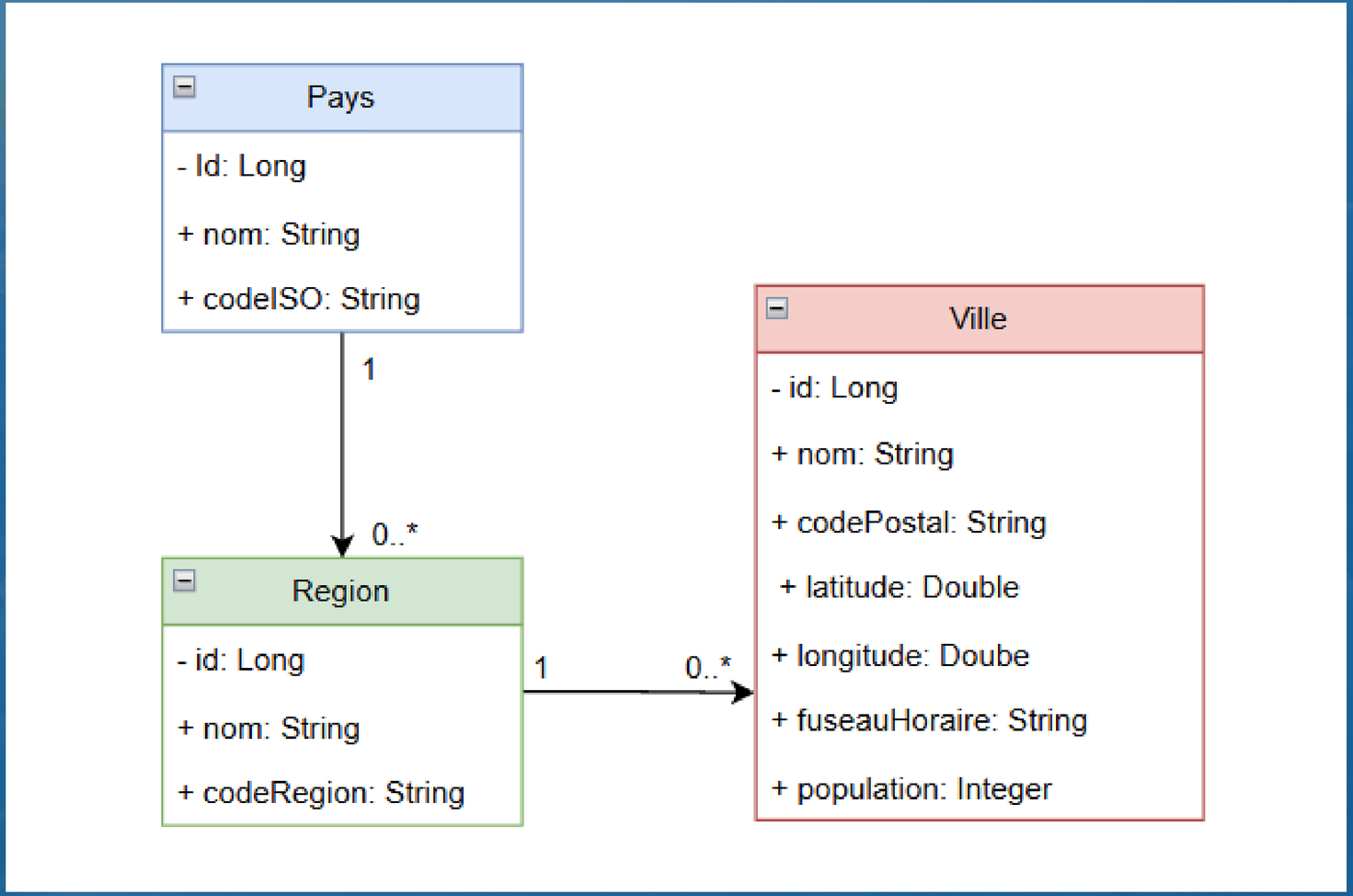
CHOIX DE LA BASE DE DONNÉES

MongoDB

ville_db

Base NoSQL adaptée à une gestion flexible et évolutive des données géographiques.

DIAGRAMME DE CLASSE
micro service Ville



API Endpoints

GET `/villes`
Liste des villes

GET `/villes/{id}`
Consulter une ville

POST `/villes/{id}`
Modifier une ville

PUT `/villes/{id}`
Modifier une ville

DELETE `/villes/{id}`
Supprimer une ville

Codes principaux

- 200 — Succès
- 201 — Ressource créée
- 204 — Suppression réussie
- 404 — Ressource introuvable

INTERACTIONS AVEC LES AUTRES MICROSERVICE



Conception du microservice Prévision



Quelles actions effectuer sur les prévisions ?

Quelles actions effectuer sur les prévisions ?



Consulter

les prévisions des 7 prochains jours



Gérer

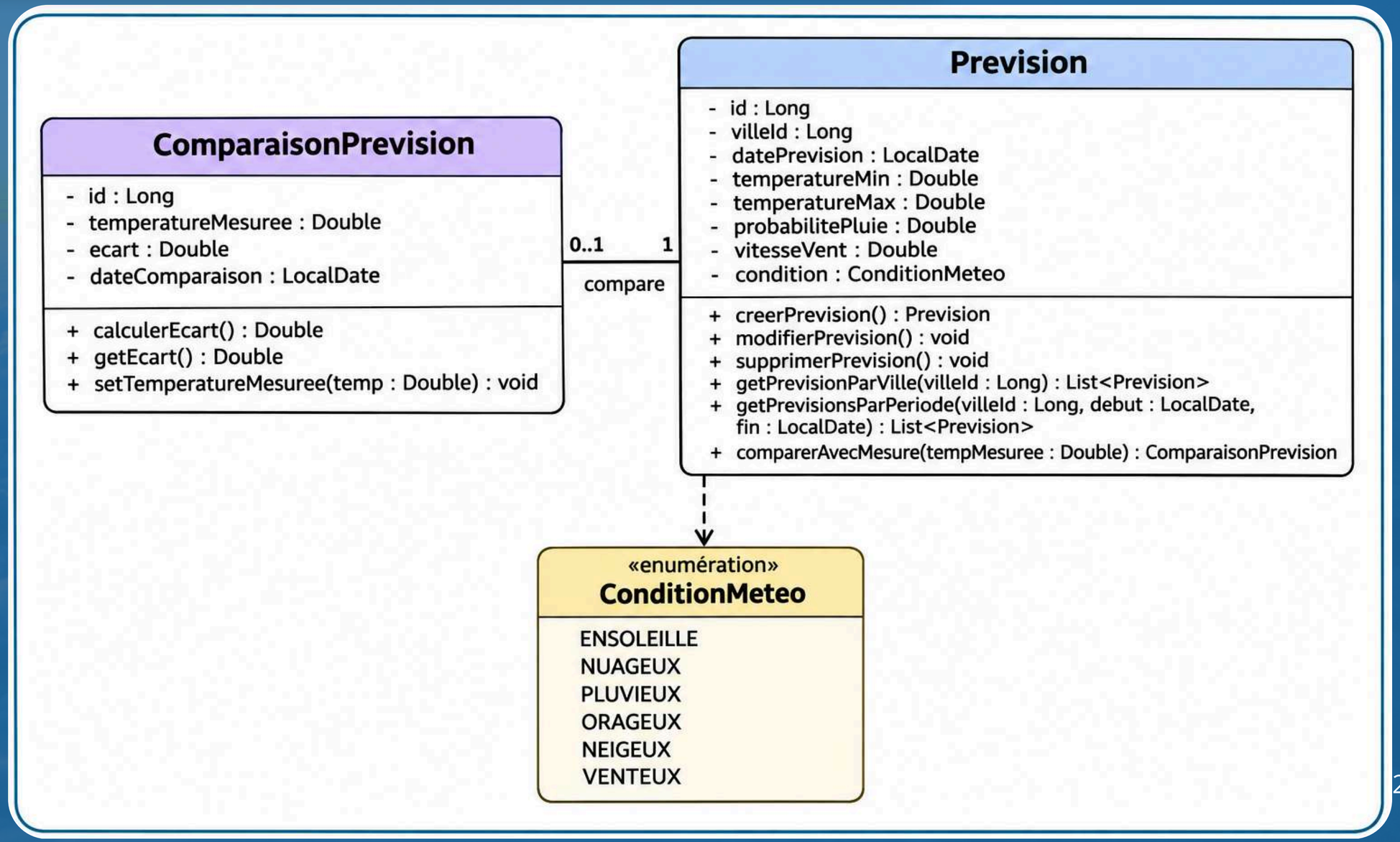
ajouter, modifier et supprimer les prévisions



Comparer

les prévisions avec les mesures réelles

Diagramme De Classes - Prévisions



Interactions avec les autres microservises – Prévision

SYNCHRONE · REST / OPENFEIGN

Ville
vérifier l'existence de la villeId (ex : Tunis)

Relevé
récupérer la mesure réelle à comparer

Station indirect via Relevé · températures

Prévision Service
Prévisions · Comparaisons
PostgreSQL

ASYNCHRONE · BROKER

Alerte
PREVISION_CREATED / UPDATED → alerte prévisionnelle

Ville
VILLE_DELETED → supprimer les prévisions de la ville

EXEMPLE

1. **Prévision : 23 °C à Tunis**
2. **Relevé : 25 °C mesurés**
3. **Écart calculé : 2 °C**
4. **Comparaison enregistrée**

JUSTIFICATION DU CHOIX DE POSTGRESQL

Microservice Prévisions | Base de données relationnelle

DONNÉES STRUCTURÉES	Prévisions, températures, dates et comparaisons : données organisées en tables.
INTÉGRITÉ & FIABILITÉ	Transactions ACID et cohérence entre prévisions et comparaisons.
RECHERCHES EFFICACES	Filtrer les prévisions par ville et par date grâce aux index.
INTÉGRATION SPRING BOOT	Compatible avec Spring Data JPA et Hibernate.
OPEN SOURCE	Solution gratuite, robuste et sans coût de licence.

CHOIX RETENU : une base fiable, structurée et adaptée aux besoins des Prévisions.

Contrat d’API – Prévision

MÉTHODE	ENDPOINT	ACCÈS	CODES DE RÉPONSE
GET	/api/previsions	Public	200
GET	/api/previsions/{id}	Public	200, 404
GET	/api/previsions/ville/{villeId}/7jours	Public	200
POST	/api/previsions	ADMIN	201, 400, 401, 403
PUT DELETE	/api/previsions/{id}	ADMIN	200 / 204, 400, 401, 403, 404
POST	/api/comparaisons	USER	201, 400, 401

GESTION DES PANNES

Microservice Prévisions | Comportements prévus

VILLES	Service REST indisponible	**Création suspendue ; prévisions existantes consultables.**
RELEVÉS	Mesures réelles inaccessibles	**Comparaison temporairement désactivée ; consultation maintenue.**
ALERTES	Réception des événements impossible	**Enregistrement maintenu ; événements à retransmettre si persistés.**
STATIONS	Panne d'un service en amont	**Pas d'appel direct ; données réelles potentiellement indisponibles.**

MÉCANISMES DE RÉSILIENCE ENVISAGÉS

TIMEOUT
Limiter l'attente

RETRY
Réessayer avec limite

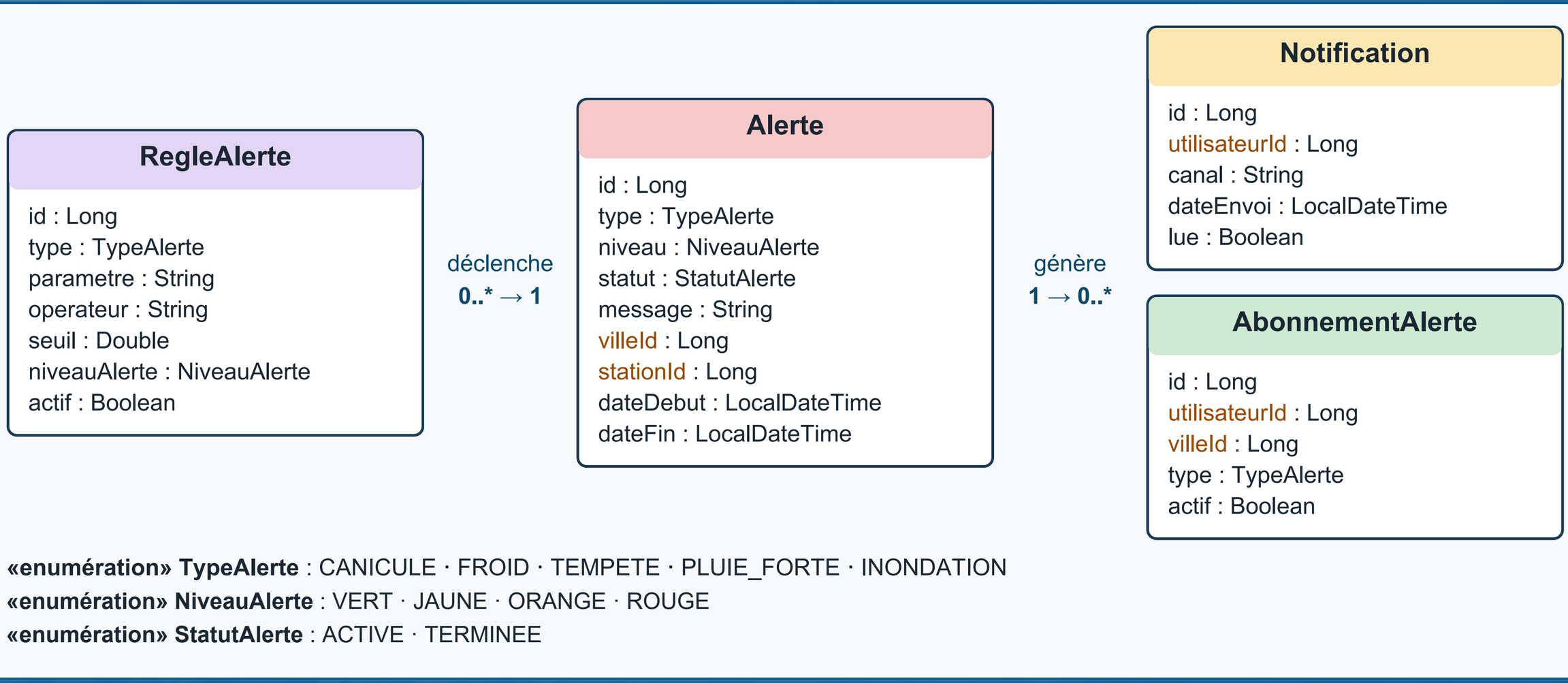
CIRCUIT BREAKER
Isoler un service en panne

FALLBACK
Conserver un service partiel

Conception du microservice Alerte



Diagramme de classes – Microservice Alerte



«enumération» TypeAlerte : CANICULE · FROID · TEMPETE · PLUIE_FORTE · INONDATION
«enumération» NiveauAlerte : VERT · JAUNE · ORANGE · ROUGE
«enumération» StatutAlerte : ACTIVE · TERMINEE

Interactions – Microservice Alerte

Asynchrone : événements consommés

Relevé
Relevee_CREATED → évaluer les règles, créer l'alerte

Station
STATION_STATE_CHANGED → état de la station

Ville
VILLE_DELETED → nettoyer règles et abonnements

Prévision
PREVISION_CREATED / UPDATED → alerte prévisionnelle

→

Alerte Service

Règles · Alertes

Notifications · Abonnements

MongoDB (alerte_db)

→

Synchrone : REST / OpenFeign

Auth
valider l'utilisateurId d'un abonnement

Ville
valider la villeId d'une règle ou d'une alerte

Asynchrone : événement publié

→

Notifications
ALERTE_CREATED → notifier les abonnés

Exemple

Mesure : 42 °C à Tunis → **Règle Canicule : seuil dépassé** → **Alerte créée : niveau ROUGE** → **Notifications aux abonnés de Tunis**

Contrat d’API – Microservice Alerte

Endpoint	Méthode	Accès	Codes de réponse
/api/alertes/actives	GET	Public	200
/api/alertes	POST	ADMIN	201, 400, 401, 403
/api/regles-alerte	GET / POST	ADMIN	200 / 201, 400, 401, 403
/api/regles-alerte/{id}	PUT / DELETE	ADMIN	200 / 204, 401, 403, 404
/api/abonnements	POST / DELETE	USER	201 / 204, 400, 401, 404, 409
/api/notifications	GET	USER	200, 401

Événements consommés

MESURE_CREATED · STATION_STATE_CHANGED · VILLE_DELETED · PREVISION_CREATED · PREVISION_UPDATED

Événement publié

ALERTE_CREATED

Gestion des pannes — Microservice Alerte

Impact des indisponibilités des autres microservices et comportement attendu

SERVICE EN PANNE	IMPACT SUR ALERTE	COMPORTEMENT PRÉVU
AUTH	Impossible de valider l'utilisateurId d'un abonnement.	Refuser les nouveaux abonnements ; conserver les alertes et règles existantes.
VILLE	Impossible de valider la villeId d'une règle ou d'une alerte.	Différer la création de règles ; les alertes déjà créées restent consultables.
RELEVES	Plus de nouvelles relevés à évaluer.	Garder les alertes actives consultables ; traiter les événements au retour du service.
NOTIFICATIONS	ALERTE_CREATED ne peut pas être traité.	Enregistrer quand même l'alerte ; l'événement reste en file d'attente.



Microservice Station

Microservice Station
Rôle et responsabilités

Rôle et responsabilités

Objectif	Assurer la gestion des stations météorologiques et de leurs capteurs au sein de la plateforme SkyCast.
Responsabilités principales	• Gestion des stations et de leurs capteurs. • Localisation géographique et rattachement aux villes. • Suivi de l'état des stations : active, inactive, maintenance. • Mise à disposition des informations aux autres microservices.

Architecture technique Microservice Station

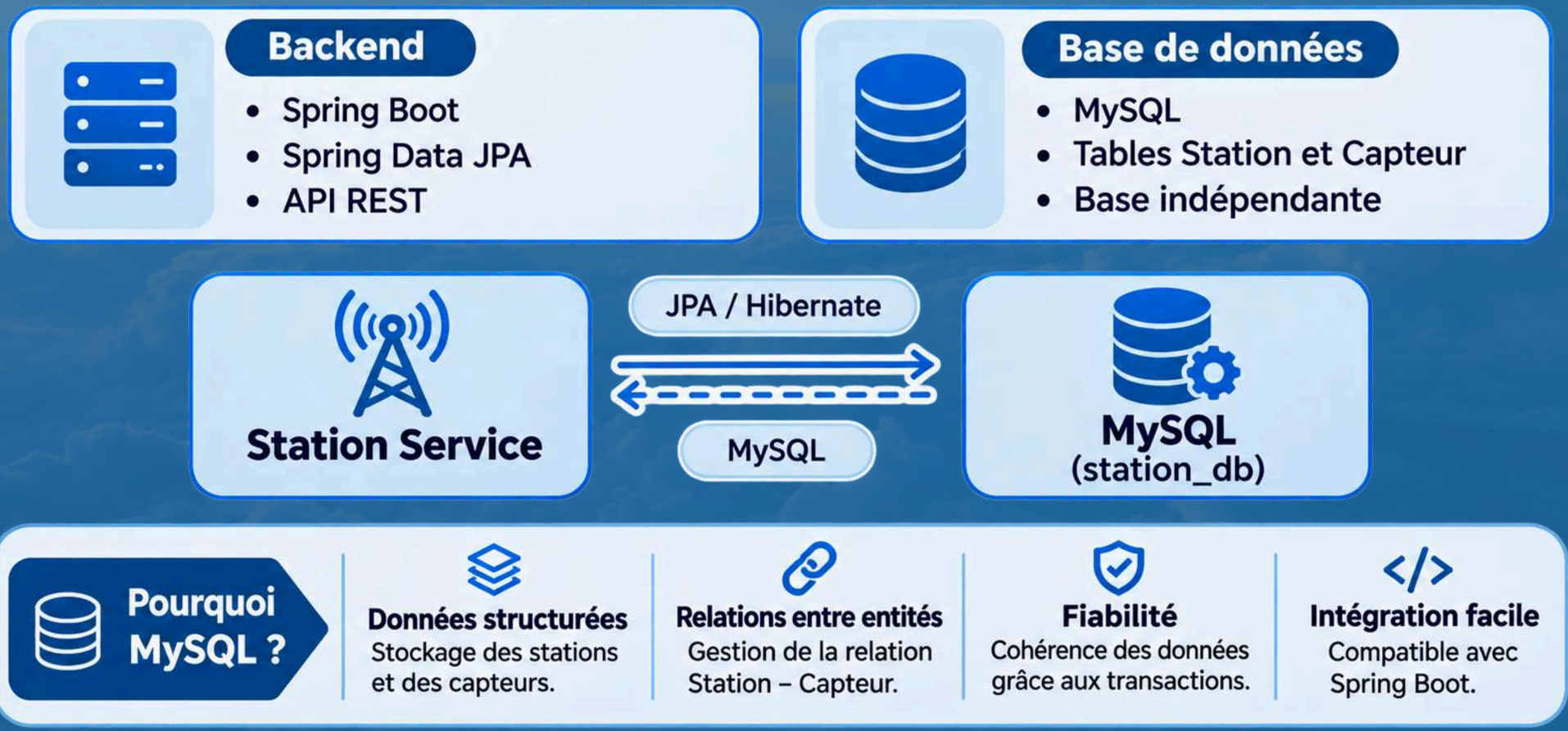
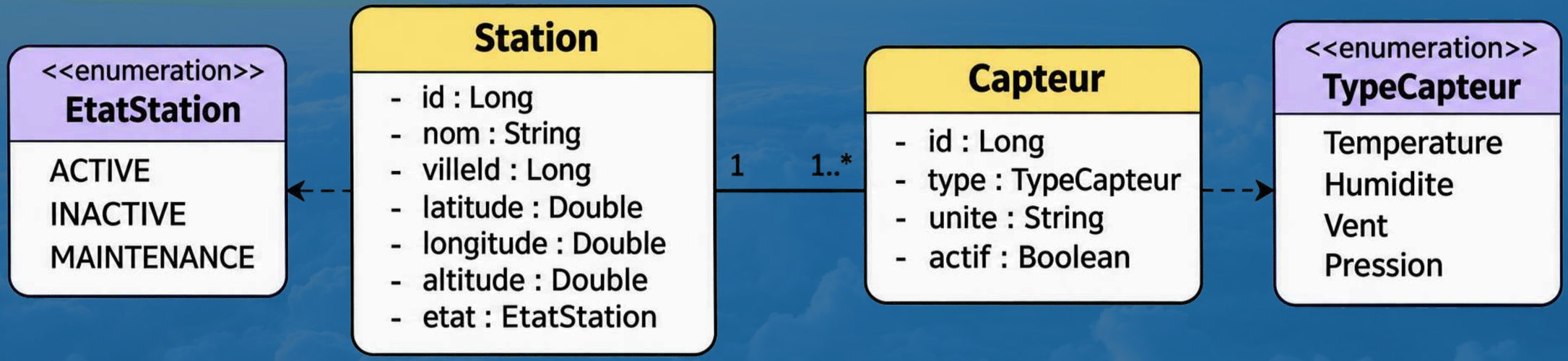


Diagramme de classes
Microservice Station



API REST — Station Service

POST	/api/stations	Admin	201 Created
GET	/api/stations	Public	200 OK
GET	/api/stations/{id}	Public	200 OK / 404 Not Found
PUT	/api/stations/{id}	Admin	200 OK / 404 Not Found
DELETE	/api/stations/{id}	Admin	204 No Content / 404 Not Found

Interactions – Microservice Station



Synchrone (REST)
Vérification de la ville et de la station.

Asynchrone (événement)
StationCreated : publié après la création d'une station (pour informer les services abonnés, ex. Relevé Service).

Données échangées

- **Vers Ville Service :** villeId
- **Vers Relevé Service :** stationId
- **Événement :** StationCreated { id, nom, villeId, latitude, longitude }

Gestion des pannes

- **Ville Service indisponible** : timeout / HTTP **503** → création de station temporairement refusée.
- **Station Service indisponible** : timeout / **HTTP 503** → vérification des stations impossible pour Relevé Service.

Conception du microservice Relevés



Microservice Relevés — Rôle et fonctionnalités

Le service Relevés centralise les observations météorologiques réelles des stations et facilite leur exploitation.

01

Gérer les relevés

Ajouter, consulter, modifier et supprimer les mesures météo.

02

Consulter les observations

Afficher la dernière mesure d’une station et l’historique des relevés.

03

Produire des statistiques

Calculer les résumés journaliers : températures min., max., moyenne et humidité moyenne.

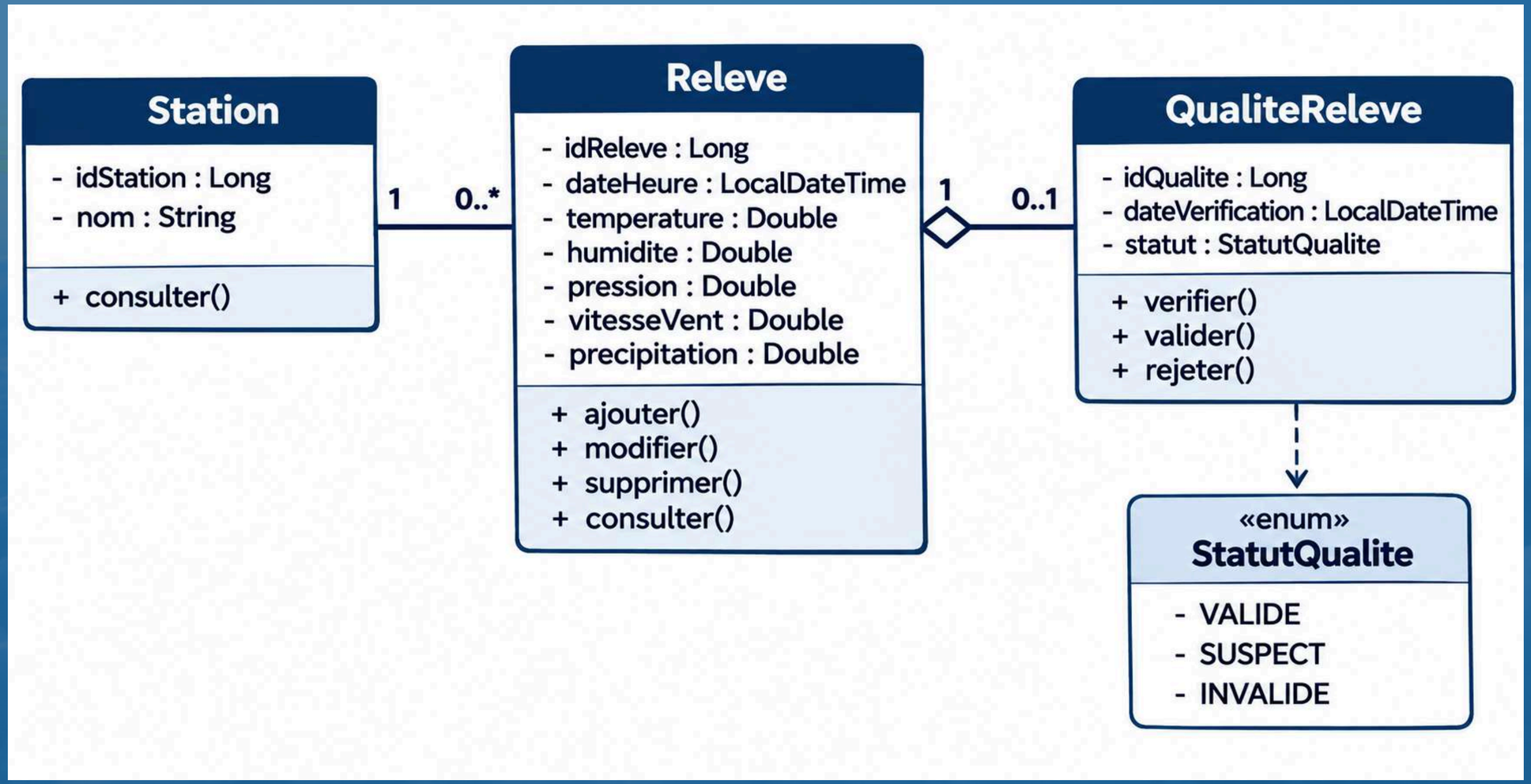
04

Contrôler et partager

Vérifier la qualité des relevés et fournir les mesures réelles au service Prévisions.

Données : température • humidité • pression • vitesse et direction du vent

Diagramme de classe



Interactions avec les autres microservises

Stations

Vérifier que la station ST01 existe

Prévisions

25 °C mesurés contre 23 °C prévus



Alertes

Température supérieure à 40 °C

Villes

Station ST01 située à Tunis

Contrat d’API — Relevés

MICROSERVICE / RELEVÉS MÉTÉOROLOGIQUES

Méthode HTTP	Endpoint	Description	Codes HTTP
GET	/api/releves	Consulter tous les relevés	200
GET	/api/releves/{id}	Consulter un relevé	200, 404
GET	/api/releves/station/{stationId}	Consulter les relevés d’une station	200
POST	/api/releves	Ajouter un relevé	201, 400, 404
PUT	/api/releves/{id}	Modifier un relevé	200, 400, 404
DELETE	/api/releves/{id}	Supprimer un relevé	204, 404
POST	/api/releves/{id}/verifier	Vérifier la qualité d’un relevé	200, 400, 404

Gestion des pannes — Microservice Relevés

Impact des indisponibilités des autres microservices et comportement attendu

SERVICE EN PANNE	IMPACT SUR RELEVÉS	COMPORTEMENT PRÉVU
STATIONS	Impossible de vérifier une station distante.	Conserver la consultation des relevés existants ; différer la validation des nouveaux relevés.
VILLES	Les informations de localisation peuvent être indisponibles.	Afficher les relevés avec stationId ; signaler les données de ville temporairement absentes.
PRÉVISIONS	La comparaison entre prévisions et mesures ne fonctionne plus.	Maintenir la collecte, l'historique et les statistiques des relevés.
ALERTES	Les notifications liées aux mesures ne peuvent pas être transmises.	Continuer à enregistrer les relevés ; mettre les événements en attente si une file est prévue.

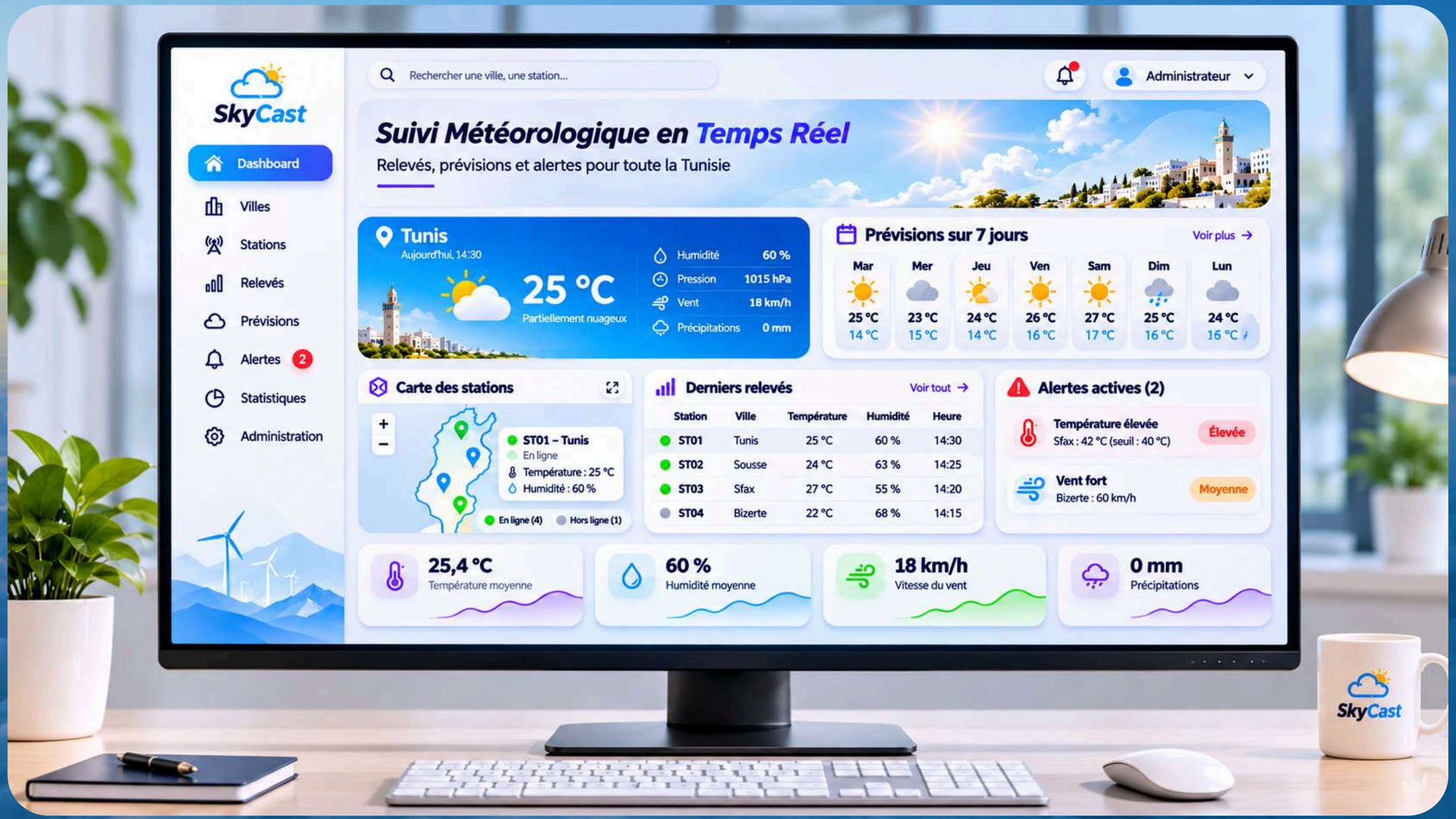
Principe : isoler les pannes et préserver les fonctions locales de Relevés.

Conclusion

SkyCast propose une solution météorologique moderne basée sur une architecture microservices, permettant une gestion flexible et une communication efficace entre les différents services pour fournir des informations météo fiables et évolutives.

Conception


SkyCast
Accueil
Villes
Stations
Prévisions
Alertes
À propos
Se connecter
S'inscrire
La météo en temps réel, partout en Tunisie
Consultez les relevés, les prévisions et les alertes météorologiques en toute simplicité.
Consulter la météo
Découvrir la plateforme
Tunis
Aujourd'hui, 14:30
25 °C
Partiellement nuageux
Humidité 60 %
Vent 18 km/h
Pression 1015 hPa
Relevés en temps réel
Mesures météo des stations
Prévisions fiables
Sur 7 jours
Alertes météo
En cas de phénomènes extrêmes
Statistiques
Analyses et tendances
SkyCast


SkyCast
Rechercher une ville, une station...
Administrateur
Dashboard
Villes
Stations
Relevés
Prévisions
Alertes 2
Statistiques
Administration
Suivi Météorologique en Temps Réel
Relevés, prévisions et alertes pour toute la Tunisie
Tunis
Aujourd'hui, 14:30
25 °C
Partiellement nuageux
Humidité 60 %
Pression 1015 hPa
Vent 18 km/h
Précipitations 0 mm
Prévisions sur 7 jours
Voir plus
Mar 25 °C 14 °C
Mer 23 °C 15 °C
Jeu 24 °C 14 °C
Ven 26 °C 16 °C
Sam 27 °C 17 °C
Dim 25 °C 16 °C
Lun 24 °C 16 °C
Carte des stations
ST01 – Tunis
En ligne
Température : 25 °C
Humidité : 60 %
En ligne (4)
Hors ligne (1)
Derniers relevés
Voir tout
Station Ville Température Humidité Heure
ST01 Tunis 25 °C 60 % 14:30
ST02 Sousse 24 °C 63 % 14:25
ST03 Sfax 27 °C 55 % 14:20
ST04 Bizerte 22 °C 68 % 14:15
Alertes actives (2)
Température élevée
Sfax : 42 °C (seuil : 40 °C)
Élevée
Vent fort
Bizerte : 60 km/h
Moyenne
25,4 °C
Température moyenne
60 %
Humidité moyenne
18 km/h
Vitesse du vent
0 mm
Précipitations
SkyCast

Merci de votre attention